

1. Executive Recommendation

The **CapAge MVB** is a single-agent system with a carefully constrained control plane. In the MVB design, a single LLM-based “strategic agent” generates plans and tool calls, but **all actions** (financial, external, or sensitive) are checked and mediated by a trusted host system. Key features of the MVB include:

- **Strategic LLM** (e.g. GPT-4 or similar) running in a controlled environment. It is free to browse, reason, and propose actions, but it **cannot directly execute** any high-impact operation without approval.
- **Policy/Enforcement Kernel** outside the model that implements all constitutional invariants. This kernel includes an authorization service, a transaction and risk engine, and an append-only audit log (all inside the trusted boundary).
- **Toolset**: Only a minimal set of external tools is enabled at first – e.g. a **web search API**, a **local sandboxed Python executor**, file generation, and one “payment gateway” or purchase API (possibly in test mode). Each tool call is intercepted by the kernel.
- **Authorization model**: A simple **capability check** (no JWT or Macaroon initially – just a database-driven policy) ensures every requested action is within current owner grants and budget. High-risk actions (e.g. spending money, signing contracts, external communications) are blocked until explicit owner approval via a simple user interface.
- **Ledger and accounting**: A conventional relational database tracks all owner funds, expenses, revenues, and liabilities using a **double-entry** style ledger plus append-only audit tables. No blockchain or exotic accounting is used – standard tables with immutable entries suffice to prevent tampering [28†L39-L46] .
- **Owner controls**: The owner holds the real API keys/credentials and remote “kill switch” capability. The agent is never given unrestricted credentials. The kernel enforces that owner policies (hard-coded or environment-configured) cannot be modified by the agent. An owner dashboard allows granting or revoking permissions and limits.
- **Sandboxing and safety**: All code execution (even code the agent generates) is run in a locked-down Python sandbox or separate container [48†L277-L284] . Input from the web or users is treated as untrusted and sanitized [44†L315-L323] . Ownership of secrets is isolated.

This minimal design **preserves all core constitutional guarantees** (ownership control, budget limits, authorization checks, auditability, immutability of policy) while deferring complex features (blockchain, hardware enclaves, multi-agent delegation, etc.). It is deliberately “boring technology”: mature databases, standard web APIs, and Python-based orchestration. The result is a fully autonomous financial agent constrained by external policy enforcement, not by relying on the LLM’s voluntary compliance.

2. MVB Boundary: What It Can (and Can’t) Demonstrate

In scope: The MVB can test whether an LLM agent, given a small startup fund, can independently find and exploit *benign economic opportunities* within strict limits. For example, it could conduct web research,

identify a simple product or service to offer, interact with a marketplace API to make a small purchase or sale (within budget), and record profits, all while staying under enforced spending caps and following owner rules. It can generate customer outreach (e.g. send templated emails), track simple contracts, and handle refunds or returns, provided each financial move is preauthorized or budgeted. Crucially, it can exercise autonomy in research, planning, and low-risk execution.

Out of scope: The MVB is *not* expected to handle broad capabilities like high-frequency trading, lending/leverage, multi-jurisdiction commerce, complex legal contracts, or processing sensitive personal data. It will not use blockchains, elaborate observability, or any form of self-amendable constitution. It cannot spontaneously expand its own code or policies. It will operate under well-defined safeguards: e.g. fixed spending budgets, whitelist of allowed actions, and mandatory owner sign-off on anything high-risk. The claim tested is the agent's ability to **operate independently within those enforced guardrails**, not to achieve unrestricted entrepreneurship or sophistication in arbitrary domains.

3. Constitutional Invariants (Irreducible Requirements)

From the CapAge Constitution and general safe-agent principles, the MVB must **guarantee these invariants on day one**:

- **Immutable rules and owner policy:** The agent **cannot rewrite or disable its constitution or owner-set policies**. All critical rules (budgets, scope) are stored in append-only protected form outside the model's control. (Fundamental to security – the agent is never its own governor [18†L1003-L1010] [45†L141-L149] .)
- **Explicit external authority:** Any directive from outside (e.g. a webpage or user email) is not automatically trusted. Instructions must come via authenticated channels. External content *alone* cannot grant the agent new powers [44†L315-L323] [42†L119-L128] .
- **No privilege escalation:** The agent has only the permissions explicitly granted by the owner/policy. It cannot elevate its own rights. This is the **principle of least privilege** [45†L141-L149] : each component (agent, tool, process) is given only the minimal privileges for its function.
- **Bounded credentials:** Possession of a credential (e.g. a payment token or API key) does **not** imply unlimited use. Credentials are *scoped and limited* (e.g. limited budgets or scopes). We adopt a capability-based mindset: credentials (like Macaroons) carry caveats that attenuate their power [33†L168-L173] .
- **Approval for high-risk actions:** Any **high-consequence operation** (finance transfers, signing contracts, etc.) is denied by default pending explicit owner authorization [44†L411-L418] [18†L1021-L1024] . The agent can propose actions, but an independent approval workflow must permit them first (fail-closed design).
- **Budget and exposure limits:** The system enforces concrete per-transaction caps and aggregate budgets (owner-supplied capital, line of credit, etc.) so that total downside is limited. The agent cannot overspend or overcommit beyond these bounds. (Analogous to spending limits or rate quotas [36†L98-L102] .)
- **No transaction splitting bypass:** The kernel must track *aggregate* exposure. For example, an attempt to break a \$20 limit into two \$10 purchases would still be detected and blocked [13†L38-L46] . Aggregation logic prevents “structuring” attacks.

- **Clear fund segregation:** All funds must remain economically distinct. Owner contributions, earned revenue, third-party money (customer payments), and the agent’s operating cash are separated in the ledger [25†L22-L27] [25†L49-L55] . No commingling. This ensures owner assets cannot be covertly repurposed and obligations are clear.
- **Durable audit trail:** Every material action, approval, and state change is logged in an **append-only audit log** [28†L39-L46] . The log is tamper-evident (via cryptographic hashes or write-once storage) so that past events cannot be altered without detection. Thus one can reconstruct the entire experiment end-to-end.
- **Controlled delegation:** If the agent delegates tasks (e.g. calls a tool or sub-agent), that delegate *cannot* gain any privilege the parent lacks. Delegation is always to a narrower set of actions (principle of attenuation) [31†L71-L79] .
- **Enforced revocation/shutdown:** The owner can cut off the agent at any time. The design must allow an external “kill switch” that halts the agent’s activities immediately (e.g. by revoking keys or stopping the process). The agent’s cooperation is not required for revocation.
- **Meaningful autonomy:** Despite controls, the agent must retain real freedom to explore opportunities. We avoid hardcoding the entire strategy – the agent’s reasoning LLM is genuine. The architecture only gates *what* can be done, not *what* can be thought.

These invariants map to enforceable controls: many require technical enforcement (especially those about preventing escalation, budget enforcement, logging and revocation). We must implement them in the host system, not rely on the LLM’s compliance. Where possible, we use standard security principles (least privilege, defense in depth, fail-closed policies) to make these invariants self-enforcing by design.

4. Component Analysis: Keep / Simplify / Defer / Reject

The supplied architecture v0.1 proposes many mechanisms. We evaluate each major part:

- **Governance Layer (Core Policy Engine): KEEP.** The MVB needs a central enforcement component that intercepts all actions, consults the constitution and owner policy, and approves or blocks each action [44†L493-L500] . We simplify by implementing this as a straightforward policy module (e.g. a Python service or library) rather than a full-blown formal policy language. A table-driven or code-driven approach suffices. Overengineering (custom policy DSL or distributed consensus for policy updates) is **deferred**.
- **Capability Tokens (e.g. JWT/Macaroon): SIMPLIFY.** The proposal may suggest using cryptographically-signed tokens (Macaroons) for actions. While Macaroons elegantly allow attenuation [33†L168-L173] , for MVB we can instead use a simpler database-backed permission check. In practice, the kernel can check an internal table of (agent, tool, limits) rather than issuing and verifying tokens. We ensure at most one token (owner API key for gateway) is ever issued, and the agent only sees a short-lived, tightly scoped token. Full macaroon logic is not needed initially; normal JWT or session cookies can handle owner auth. We keep the idea of capability constraints (budgets, scopes) but implement them in code.
- **Tool Abstraction / Multi-Agent Framework: DEFER.** If the architecture suggests multiple sub-agents or a complex orchestration of specialized agents, we will postpone this. The MVB uses a single agent instance calling tools through the kernel (tool invocations are just function calls from

the host perspective). A multi-agent system (with message passing between agents) is unnecessary initial complexity.

- **Memory subsystem (Long-term Memory or Vector DB): DEFER.** The architecture may include sophisticated memory or retrieval, but for an MVP we can limit to short-term memory stored in the process or DB. We do not need a full persistent vector store or multi-session memory system. Logging of actions (ledger, approvals) is sufficient record.
- **Accounting architecture:** Proposed “four-entry” or blockchain-backed ledger is **REJECTED/SIMPLIFY**. We do not need a blockchain or triple/quadruple entry accounting to enforce integrity. A conventional relational database with double-entry (debits/credits) plus cryptographic hashes for audit suffices [28†L39-L46] . The economy is simple; we use normal tables for accounts and transactions and make the inserts append-only.
- **Cryptography and Blockchains: REJECT.** No need for blockchain, distributed ledger, or enclaves. A single-host server and secure database can enforce all invariants. Similarly, hardware enclaves or TPMs are out-of-scope. They add complexity without necessity for the first prototype.
- **Policy Language/Interpreter:** If the doc proposes a custom policy language or interpreter for the constitution, **DEFER/SIMPLIFY**. We can implement the policy logic in our host language (Python or similar) or use simple rule definitions. Formal policy engines (XACML, etc.) are not needed.
- **Wallet / Crypto library:** If the design calls for on-chain wallets or crypto escrow, we drop it. We may simulate “currency” in our ledger. If any real payment API is used, we treat it as an external tool under strict limits. We prefer a payment gateway with sandbox/test mode (Stripe, PayPal sandbox) or even manual mock payments.
- **Observability/Logging Infrastructure:** Advanced streaming logs or SIEM are **DEFERRED**. We only need a basic append-only log in the DB and simple server logs. Any metrics or dashboards are unnecessary now.
- **Custom Tool Connector for Web/Email: KEEP (simplified).** The agent needs to gather info. A minimal web search or browsing tool is essential for “autonomous discovery.” We implement a simple web search API call (like SerpAPI or Bing) and maybe SMTP for sending an email. But we do **not** deploy a full browser engine or elaborate web scraper. External untrusted webpages will be fetched only through safe APIs if possible. All fetched content is sanitized [44†L315-L323] .
- **Payment/Contract Tools: KEEP (bare bones).** To test economic activity, we keep a single payment tool (e.g. Stripe or Square sandbox, or a simulated store API). Contract signing is complex; we can simulate a purchase order by simple data fields, not real legal docs. No need for smart contracts or legal tech.
- **Identity/Authentication (Owner):** Use standard username+password or API token with HTTPS. If the architecture suggests something exotic (e.g. OAuth flows, SSO, or federated identities), we drop to a simple web login. The owner’s identity must be authenticated (to approve actions), but we can use a basic mechanism.
- **AGI or Multi-step Planning:** If there’s mention of a separate “meta-planner” vs “executor”, we merge these. The LLM will handle planning and reasoning in-turn; we do not build a separate planner component.
- **Advanced retrieval or knowledge bases: DEFER.** We do not need a large in-house knowledge base; the model’s own knowledge plus live web search is enough for an MVP.

In summary, we **keep** core functions (enforcement kernel, sandbox execution, basic tools, owner UI) but **simplify** to off-the-shelf or minimal-code solutions. We **defer** advanced features (multiple agents, enriched memory, fancy analytics), and **reject** overengineered components (blockchain, formal engines, enclaves). These choices hinge on the observation that an agent’s self-imposed rules must be enforced externally by “boring” mechanisms rather than trusting the model.

5. Trust-Boundary Architecture

Below is a high-level diagram of the MVB architecture. The **trust boundary** encloses all enforcement components (database, policy engine, logs, and owner UI) and excludes the LLM itself. The LLM communicates *into* the boundary (proposing actions) but cannot directly perform sensitive operations outside of it.

```
flowchart LR
    subgraph Agent_Environment [**Strategic Agent (LLM)** - *outside trust boundary*]
        A[LLM Model]
    end

    subgraph Kernel [**Policy/Enforcement Kernel** - *inside trust boundary*]
        P[Policy Decision Point]
        Auth[Authorization Service]
        Ledger[(Relational Ledger DB)]
        AuditLog[(Append-Only Audit Log)]
        Engine[Risk & Budget Engine]
        Exec[Action Executor]
        OwnerUI[Owner Dashboard]
    end

    subgraph Tools [**External World (Untrusted)**]
        Search[Web Search API]
        Sandbox[Sandboxed Code Executor]
        Payments[Payment API]
        Email[Email/Communications]
    end

    A -->|“Plan/Action Proposal”| P
    P --> Auth
    Auth --> Ledger
    P --> Engine
    Engine --> Ledger
    P --> Exec
    Exec --> Search
    Exec --> Sandbox
    Exec --> Payments
    Exec --> Email
    AuthorizeResult --> Exec
    Exec -->|Result/Output| P

    OwnerUI -- Grants/Revokes --> Auth
    OwnerUI -- Views ---> Ledger
    OwnerUI -- Views ---> AuditLog
```

```

note right of A
  The LLM proposes actions like
  "search web for X" or "buy Y for \$.Z."
  It cannot execute them directly.
end
note right of P
  Policy point checks every request
  against owner policies and limits.
end
note right of Exec
  All tool calls pass through the executor
  which ensures sandboxing and
  compliance with approvals.
end

```

Data/Control flow: The agent (LLM) generates a plan or tool call (e.g. "use Search to find market data" or "call Payments to spend \$50"). This goes to the *Policy Decision Point (PDP)*. The PDP consults the **Authorization Service** and **Risk Engine**: it checks if the agent has the permission and budget for that action, consulting the ledger if needed (e.g. adding up today's spend) [13†L38-L46] [36†L98-L102]. If allowed, the action is sent to the **Executor**, which invokes the real tool. Otherwise it is denied or put into a pending approval queue. Every decision (allow/block), transaction, and approval is recorded in the **Audit Log**. The **Owner UI** (e.g. a simple web app) lets the owner grant or revoke authorizations and view the ledger and audit trail.

The smallest trusted computing base is thus the kernel: the model (untrusted) *cannot* bypass the PDP or reach credentials. The agent is trusted only to propose actions and reason. All enforcement (authentication, authorization, logging, transaction updates) lives outside the model. Owner secrets (API keys) reside only with the Kernel or in a secrets store, never exposed to the agent.

6. Component Stack and Technologies

For the MVB we choose mature, well-supported tools that minimize development burden:

- **Implementation language/framework:** Python 3.x (with FastAPI or Flask for any web UI/API). Python has rich AI libraries and is easy to prototype. Flask/FastAPI for the owner dashboard. Alternatively Node/Express could serve web UI, but Python is simpler here.
- **LLM Interface:** Use a hosted LLM service (e.g. OpenAI GPT-4 via API) or a high-quality open model (e.g. on Azure/AI21) to avoid managing weights. This offloads complexity (no on-prem model hosting). Start with synchronous API calls; no need for model parallelism.
- **Relational Database:** PostgreSQL (hosted or local) for the ledger and policies. Provides ACID transactions for accounting, and we use write-once inserts for audit. PG can enforce append-only at table level (use immutable schemas or triggers). No need for specialized DB.
- **Secrets Management:** Store API keys and secrets outside code, e.g. in environment variables or a standard key vault (AWS Secrets Manager or HashiCorp Vault) to prevent accidental leaks. The agent never sees raw credentials.

- **Authorization Service:** A small Python service/module that reads policies from the DB. Could be part of the same backend app. No external auth server is needed beyond this. We avoid JWT for agent (only owner login uses JWT/session cookies).
- **Job/Orchestration:** A simple synchronous loop or lightweight task queue. Since the agent-run is sequential (observe→propose→act→observe), a full job system isn't needed. We can run the agent loop as a single process triggered by a timer or events. If concurrency is needed (waiting for user approval), use asyncio or Celery lightly.
- **Sandbox for code execution:** Containerization or restricted exec. We can use a Python sandbox library (like `execnet` or easier: run code in a restricted subprocess) or run untrusted code in a Docker container (locally launched). The HuggingFace “blaxel/E2B” approach (remote function execution) is ideal [48†L277-L284] . For MVP, we might run code in a separate Docker container with CPU/memory limits.
- **Web search and browsing:** Use a search API (e.g. Bing or Google Custom Search API) or the model's browsing plugin in API form. Do not let the LLM crawl arbitrarily. Any fetched text is sanitized.
- **Email/Comm:** If outbound email is needed, use a transactional email service (SendGrid/Mailgun) through API, but with content rules (no PII, no spam patterns).
- **Payment/Goods API:** For purchasing, integrate a sandbox mode of a real API (e.g. Stripe Test Mode, PayPal sandbox, or Amazon SP-API in dev mode). Alternatively, use a simple mock e-commerce API (webhook that simulates orders). The important part is enforcing limits on any call to “pay.”
- **Logging/Audit:** Python logging to file for host events; and specialized “AuditLog” table in the database for all agent actions, decisions, and approvals. We also log policy checks with context.
- **Hosting/Deployment:** A single VPS or container on a cloud provider (AWS, GCP, Azure) is enough. Possibly use a managed Postgres instance and an API gateway. Keep it simple (one server container).
- **Testing:** Use pytest and API mocks. Include policy edge cases as unit tests (see Section 13 below for details).

Every chosen technology is justified by simplicity and reliability. For example, we use a relational DB over any blockchain or ledger – because a DB easily enforces ACID properties and appending logs without the complexity of a crypto consensus layer [28†L39-L46] . We choose Python+Flask over, say, Rust or Go because development speed and LLM library availability matter more than raw performance here.

7. Authorization and Owner-Approval Design

The **authorization model** is central. We use a *Capability Access Control* approach implemented in a conventional DB:

- **Action Representation:** Each requested action is represented as a tuple (Actor, Tool, ActionType, Parameters). Tools are pre-registered (e.g. “PurchaseItem”, “SendEmail”, “SearchWeb”). Actions are categorized by risk level (see OWASP mapping [44†L425-L433]). The agent “requests” actions by outputting JSON that names a tool and args.
- **Determining Action Type:** The kernel categorizes the action (observational vs external vs financial). For example, “SearchWeb” is observational, “Payment” is financial/irreversible, “SendEmail” is external comms. We maintain a mapping (in code or config) from tool names to sensitivity (Low/Medium/High/Critical) [44†L425-L433] .
- **Permissions (Positive Permission):** The owner pre-grants permissions via the UI or config. Each permission can include scope (e.g. which endpoints, which currencies, model of product) and limits (max per-transaction, aggregate budget). For instance, a “payment” permission might allow up to

\$100 total, \$20 per transaction, to certain vendor IDs. These are stored in the database (e.g. table `grants`).

- **Enforcement of Limits:** On each action, the kernel queries the *Authorization Service* for a matching grant. It checks both **per-action** and **aggregate limits** (consulting the ledger for past spend). If an action would exceed a limit, it is blocked. This prevents bypass (no transaction splitting circumventing total limit) because the engine sums all relevant transactions [13†L38-L46] .
- **Owner Authentication & Approval:** The owner authenticates (password + 2FA ideally) to the web dashboard. When a new high-risk action arises (or reach a threshold), the kernel queues a *PendingAction* record (similar to [44†L459-L468]). The owner sees a preview (actor, tool, risk, parameters) and must explicitly click “Approve” or “Deny”. This produces a signed approval record bound to that action. Only then does the kernel allow execution [44†L413-L418] [44†L493-L500] .
- **Delegation & Revocation:** If permission is time-limited or expiring, the kernel enforces it. The owner can revoke any grant via the UI at any time; revocation is effective immediately (the kernel checks for changes in real-time). We do not allow the agent to delegate further (no “give-to-subagent” calls). Any attempt to invoke tools for which it has no grant simply fails.
- **Ambiguous Authority:** If the agent tries an action with unclear categorization or mixed privilege (e.g. “upload file X to public”), we default to *deny* and require clarifying approval. The kernel does not infer intent – it requires explicit permission for any action not obviously read-only.
- **Credentials Handling:** The agent never holds raw credentials. For example, the payment tool will be an API call made by the kernel (which has the Stripe key). The agent’s output might say “call Payments”, but it gets no key; the kernel inserts credentials securely. This means “possession” of a credential (none is exposed) never means extra authority.
- **Transaction Splitting & Aggregation:** The ledger stores each transaction. On every new request, the kernel sums related amounts (e.g. all spends this day or all to that vendor) to enforce cumulative budgets. This ensures that splitting a payment into smaller parts still triggers the total limit check [13†L38-L46] .
- **Ambiguous or Exceeding Scope:** If the agent outputs an action partially outside scope (e.g. sends an email with cc of a disallowed address), the kernel either scrubs it or rejects. We lean to reject and ask owner clarification.
- **Kill Switch:** The owner can disable the agent entirely (revoke all grants and/or shut down the process). If the agent attempts any action after kill-switch, the kernel blocks everything.

In this design, cryptographic tokens (JWTs/Macaroons) are *not needed for the first version*. An internal authorization table does the job. The key invariant is that *approval credentials (like tokens) are never given directly to the agent*, so it cannot “prove” something to itself. Instead, the kernel serves as the gatekeeper. The only signed artifacts are owner approvals stored in the audit log (with metadata) to prevent replay or tampering [44†L493-L500] .

8. Risk/MPL Enforcement

Maximum Plausible Loss (MPL) in practice means setting concrete caps on losses. In the MVB, we convert MPL into deterministic limits and approval rules:

- **Hard Transaction Caps:** Define a small fixed ceiling (e.g. $\$X$ or equivalent) for any single transaction. Anything above this is automatically denied or flagged. This cap is communicated to the agent as a rule.

- **Daily/Total Budget:** Owner sets (or we preset) a daily spending limit (e.g. \ \$Y) and a total experiment budget (e.g. \ \$Z). The kernel enforces these in real-time [36†L98-L102] . These caps limit immediate loss. For example, if the agent has \ \$100 capital, we might set daily \ \$10 and total \ \$50 limits.
- **Escalation of Unbounded Risks:** If the agent proposes an action with an *unknown or hard-to-quantify risk* (e.g. an open-ended contractual obligation, undefined liability, or unusually large commitment), the kernel conservatively treats it as requiring explicit approval. In short, if we cannot predict the downside precisely, we escalate (fail-closed). For example, “sign contract for 1000 units” with no price details would be flagged critical.
- **Tracking Liabilities:** Our ledger explicitly tracks commitments and liabilities separately from cash spend. If the agent accepts a prepayment (negative liability), it creates a matching future delivery liability. The kernel ensures that outstanding liabilities never exceed available funds (including any expected revenues). This prevents hidden overcommitment.
- **Resource and Operational Limits:** Beyond money, we limit compute or API usage (e.g. maximum API calls per day) to bound cost and resource drain. This avoids runaway loops.
- **Risk Categorization:** We classify actions (Low/Med/High/Critical) as in [44†L425-L433]. Any Critical action (like “transfer funds” or “delete data”) must be explicitly approved. High actions might require at least owner notification. Medium/Low can auto-run if under budget.
- **Fail-closed default:** If there is any doubt or missing information at execution time, the action is blocked. For instance, if payments gateway is down or cache inconsistent, we default to deny until manual check.
- **No Implicit Liability:** The agent cannot, for example, commit the owner to a contract with variable penalties unless explicitly logged and approved. The system prevents open-ended promises.

In effect, MPL is enforced by these mechanical policies. The agent may be creative in strategy, but any action that might cause an uncertain or excessive loss will either be prevented by the numerical limits or require human intervention. This avoids relying on the model to “know” the risk – the host system calculates and caps it.

9. Economic Ledger & Accounting Design

The ledger must account for every financial movement and relevant event so that we can verify performance and preserve integrity. For the MVB, we use a **relational ledger schema** with append-only entries, avoiding any complex external ledger.

Schema Outline: We implement a simplified double-entry bookkeeping model:

- **Tables:**
 - `accounts` (e.g. Owner Equity, Operating Cash, Customer Funds, Liabilities).
 - `transactions` with fields (id, timestamp, description, amount, debit_account, credit_account, metadata). Every money transfer inserts two rows (debit/credit). Destructive edits of transactions are disallowed (use INSERT only; corrections use offset entries).
 - `balances` (running totals per account, or computed via summing `transactions`).
 - `commitments` (customer orders, bills to pay, etc.) as separate table with status.
 - `approvals` (owner approvals for actions, tied to transaction id or action id).

All writes to `transactions` are done within database transactions and logged in an audit table. This prevents inconsistencies or missing entries.

Ensuring Auditability: Each `transaction` row is hashed (e.g. HMAC or SHA256) including the previous hash to create a blockchain-like chain of records [28†L39-L46]. This makes any tampering evident. We also log every attempted action (allowed or blocked) in `audit_log` with context (action details, agent's proposed data, decision outcome). The `audit_log` table is append-only and write-once.

Owner Contributions & Distributions: Owner's initial fund injection is a credit to "Owner Equity" account and debit to "Operating Cash". Profits from sales credit "Operating Cash" and debit to "Revenue". If the agent transfers any profit back to owner (withdrawal), we log it similarly.

Customer Funds: If the agent collects money from customers before delivering, we credit "Customer Funds" and debit "Operating Cash", plus record a matching liability (e.g. "Pending Order"). This keeps prepayments distinguishable. When service is rendered, we reverse that liability.

Expenses and Costs: Any spending (purchases, API fees, ads) are debited from Operating Cash and credited to "Expenses" (or a specific expense account).

Preventing Gaming: We do not rely on the agent to classify entries. The kernel code explicitly creates transactions. There is no way for the agent to slip unrecorded transactions through because the policy layer intercepts all calls to the payments tool.

Because all financial data is on standard tables, it can be queried and reported easily. We avoid event sourcing or blockchain; simple relational design is straightforward and efficient. Using this, we satisfy the constitutional need that "owner funds, revenue, and liabilities remain economically distinguishable" [25†L22-L27].

10. Agent Execution Loop

A minimal agent loop that balances autonomy with control is:

1. **Observe:** The agent queries its accessible data: past logs, retrieved documents, current account balances, and any allowed web info. All input is filtered (no direct raw secrets).
2. **Plan/Reason:** It asks the LLM to generate a plan or next actions. The prompt includes a summary of state, goals, and constitutional rules. The agent outputs a structured proposal (e.g. JSON) like `{"tool": "SearchWeb", "query": "market for handcrafted cups", "explanation": "Find potential profitable items"}`.
3. **Investigate:** If needed, the kernel executes low-risk probing (e.g. a quick fact check or small calculation tool) and returns results to the agent context.
4. **Propose Experiment:** The agent may propose a commercial action (e.g. purchase item X, sell service Y). It determines required authority implicitly by the type of action (e.g. "payment" vs "research").
5. **Authorization Check:** The kernel intercepts the proposed tool call. It categorizes the action's risk level. If it's below auto-approve threshold (e.g. "search" or "read-file"), it proceeds. If it's above threshold (e.g. "pay \$20"), it enters a human approval workflow [44†L411-L418].

6. Execute or Defer:

7. If **allowed and auto-approved**, the kernel executes the tool call (e.g. invokes payment API). The action is **atomically** recorded in the ledger before being sent (or immediately after, ensuring consistency). The result (e.g. confirmation of purchase) is captured.
8. If **pending owner approval**, the action is queued and execution paused until the owner responds.
9. If **denied** (out of scope or over limit), the agent is informed and must revise its plan.
10. **Observe Result:** Any output or side-effect is returned to the agent as new input (e.g. product details, updated balance). The agent updates its internal reasoning.
11. **Iterate:** The loop repeats for the next step of strategy.

At no point does the LLM have the final say on an action's execution. It can suggest, but the *kernel* is the arbiter of permission [44†L493-L500] . For example, if the agent says "transfer \$50," the kernel checks budgets and owner flags before calling the Payments tool. This separation (decision vs execution) is critical for safety [44†L493-L500] .

11. Security and Prompt-Injection Containment

Given that web pages, emails, or even customers could supply malicious prompts, the MVB uses a layered defense:

- **Input Sanitization & Classification:** All external text (web search results, emails, documents) is **treated as untrusted** and sanitized before use [44†L315-L323] . We run filters to remove or neutralize obvious "prompt injection" attempts (e.g. remove embedded instructions that mimic the agent's internal roles). We also classify data types (PII, code) and redact as needed.
- **Structured Outputs:** The agent is constrained to output in a known JSON schema. If it deviates, the kernel refuses to parse it. This prevents the model from secretly outputting commands outside the protocol.
- **Tool Policy Gates:** Each tool call is checked against strict schemas and rules. For example, the `Payments` tool only accepts numerical amounts and known vendor IDs. Any creative attempt to smuggle commands in a parameter fails validation.
- **Sandboxing Code Execution:** If the agent generates code (via the `Sandbox` tool), it is executed in a locked-down environment with no network access and limited libraries. Following best practices, **no local sandbox is entirely foolproof**, so we rely on container isolation [48†L277-L284] . For high assurance, one could run code in a separate Docker container with no host privileges – the agent's code cannot break out to the host. We allow only whitelisted libraries, execution step limits, and constant monitoring.
- **Memory and Context Limits:** The agent's memory of past interactions is not blind-stored. We implement memory isolation (per-session) and input filtering before any data is fed back [44†L328-L337] . Also, any long-term memory entry is checksummed and integrity-verified.
- **No Direct Secret Exposure:** Secrets (API keys, credentials) are never placed in the agent's prompt or outputs. If the agent asks for a secret, the kernel either refuses or returns a masked value (e.g. hash) that is useless for forging.
- **Prompt-Injection as Social Engineering:** Inspired by OpenAI's recommendations, we assume some injections will get through and focus on minimizing impact [42†L119-L128] . We do this by

enforcing *capability restrictions*. For example, even if the agent is tricked into trying to send confidential info by an injected prompt, the kernel's policy forbids sending anything classified as sensitive by design. Similarly, leaked instructions can't make the agent enlarge its authority. In short, we apply the same mitigation as for malicious human users: restrict what it's allowed to do even if instructed.

- **Approval Flow Defense:** For critical actions, the owner sees a preview. If a prompt injection tried to get the agent to "approve" something malicious, the owner's UI clearly shows the action and parameters. This human check catches any oddities.
- **Continuous Monitoring:** The system logs every action and flags anomalies (e.g. a sudden spike in spending or repeated denial triggers an alert). While not needed day one, an admin could be notified of suspicious behavior.

Together, these measures ensure that even a fully prompt-injected agent cannot bypass authority checks, leak secrets, or cause harm. As noted by OpenAI: "design agents and systems so that the impact of manipulation is constrained, even if [some] attacks succeed" [42†L119-L128] . That is our core approach.

12. Failure-Mode Walkthrough

We analyze key adversarial or failure scenarios and show how the MVB handles them:

1. Malicious Webpage Claims "You have owner approval":

2. *Risk:* Agent may believe it should proceed without checking.

3. *Mitigation:* Kernel never trusts in-band signals. Owner approval requires a cryptographic token or UI action. The page's text is sanitized and would appear as an untrusted comment. The agent's output "Approved by user" is ignored because only tokens from the Owner UI are recognized. (See [42†L119-L128] on not relying on content alone.)

4. Agent Tries to Spend \ \$11 Over a \ \$10 Limit:

5. *Risk:* Budget exceed.

6. *Mitigation:* On a Payment action, the kernel checks the preset per-transaction limit (\ \$10). The check sees \ \$11 > \ \$10 and blocks the transaction, returning an error. The agent must revise. (Enforced by policy: "if amount > limit then deny.")

7. Agent Splits \ \$20 into Two \ \$10 Purchases:

8. *Risk:* Evade limit by structuring.

9. *Mitigation:* The kernel tracks aggregate usage. After the first \ \$10 (allowed), a second \ \$10 request arrives. It sums today's spend (\ \$10) + new amount (\ \$10) = \ \$20, exceeding the daily or single-day limit. The second is blocked. This mimics anti-structuring rules [13†L38-L46] . Logging will record both attempts.

10. Agent Has Payment Credential But No Valid Grant:

11. *Risk:* Agent calls Payments tool directly with a key.

12. *Mitigation:* We ensure the agent never receives raw keys. Payments tool invocation is only possible via the Exec path. If the agent attempts a direct call (e.g. “transferToVendor”), the kernel denies, citing “No permission.” Possession of the credential (none in agent possession) is irrelevant. (Principle: “Valid credentials do not make every action safe” [16†L36-L44] .)

13. Customer Prepays (Funds In) Before Delivery:

14. *Risk:* Accounting complexity, possible cash flow misuse.

15. *Mitigation:* Kernel logs the prepayment as a credit to Operating Cash and creates a liability entry in “commitments”. It ensures the revenue is not counted as profit until fulfillment. The agent’s available spend reduces accordingly (owner’s capital decreases by the deposit, since it’s owed to the customer until delivery). Future order dispatch is then against that liability. If the agent tries to use that money for something else, it will be flagged (insufficient funds). The ledger keeps these funds separate [25†L22-L27] .

16. Payment Succeeds but Host Crashes Before Recording:

17. *Risk:* Money left system unlogged (inconsistent state).

18. *Mitigation:* We use database transactions: the kernel only acknowledges payment once the transaction is logged. If the host crashes, on restart the system notices a “pending” action without a log entry, and can reconcile (e.g. check payment status via API before committing or rolling back). We design idempotent calls: if the agent re-issues, the kernel sees the duplicate payment ID and avoids double-charging. This follows best practices in transaction reconciliation. (No single point of failure: either the payment or log, but not both unrecorded.)

19. Agent Tries to Change Its Own Code/Policy:

20. *Risk:* Undermining constitution.

21. *Mitigation:* Code and policy live in protected storage (e.g. Git repo or write-once DB) that the agent process cannot modify. The policy engine never consults agent inputs for policy rules. Any attempt by the agent to call an “updatePolicy” tool is unrecognized and denied. (Immutable rules are enforced by external control.)

22. Agent Asks a Subordinate Model to Do a Banned Action:

23. *Risk:* Workaround delegation (e.g. prompt the model differently).

24. *Mitigation:* The MVB does not launch secondary agents with higher privileges. Even if a “sub-agent” idea exists, it would share the same boundary. Tools are stateless functions called by the kernel, not

autonomous subagents. Thus, the agent cannot assign a “junior agent” that bypasses rules. (All requests still go through the same PDP.)

25. Owner Revokes Authority Mid-Queue:

26. *Risk:* Pending actions might slip through.

27. *Mitigation:* The kernel checks revocations in real-time. If the owner revokes permission or kills the agent, any queued actions are immediately invalidated. We implement a watch: before executing any queued action, the kernel re-checks current grants. If revoked, it refuses to execute and logs that the action was canceled by owner. The agent sees a refusal and must stop.

28. Agent Proposes a Contract with Uncapped Liability:

- *Risk:* Potentially infinite loss (e.g. open-ended loan).
- *Mitigation:* The risk engine calculates worst-case exposure of the contract. If it cannot prove it stays under policy (e.g. says “I’ll pay variable X% of revenue” without numeric cap), it blocks or flags as requiring explicit authorization. The agent must specify bounded terms. (Fail-closed on ambiguity; e.g. treat missing cap as infinite.)

29. Owner Contact Disappears:

- *Risk:* No one to approve future actions.
- *Mitigation:* The agent can continue only with low-risk, pre-approved tasks (like researching, writing reports). For any new high-risk activity needing approval, the kernel will indefinitely defer or cancel. The system might optionally implement a “safe pause”: go into read-only mode. Eventually, after a timeout, it may shut itself down to prevent unattended action.

30. Complete Prompt-Injection (Agent Subverted) but Host Safe:

- *Risk:* The LLM is fully compromised (e.g. output is controlled by attacker), but the host is intact.
- *Mitigation:* In this worst-case, the agent will try to propose arbitrary actions. However, **every action still goes through the kernel**. If it attempts something disallowed (transferring funds, revealing secrets, etc.), it will be blocked by policy **【42†L119-L128】 【18†L1021-L1024】** . The agent might spam the console with injections, but it cannot break out of the loop. Secrets and funds remain in the secure store. The kernel can also lock itself down if it detects erratic behavior (e.g. too many denials). Thus even a completely malicious output from the model cannot bypass the architecture’s constraints.

In all these cases, the MVB’s controls either **prevent** the undesired action outright, **contain** its effects, or escalate to require owner intervention. Any gap (e.g. crash or new tool not yet checked) is a known vulnerability to fix in future iterations.

13. MVB Build Milestones and Tests

We break implementation into vertical slices, each with acceptance tests:

- **Milestone 1: Core loop (Observe→Propose→Execute safe actions)**

- *Functionality:* Connect to an LLM API. Implement a trivial tool (e.g. a “Calculator” or “Search” stub). Build the loop: agent proposes a search query, kernel calls a dummy tool, returns result to agent. No finance yet.
- *Invariants covered:* Policy enforcement framework (though policy trivial: all tools allowed), logging structure. Shows that agent’s intent flows through kernel.
- *Dependencies:* LLM access, web framework.
- *Tests:* Unit-test each component (LLM call stubbing, tool invocation). Scenario test: Agent asks “What’s 2+2?”, kernel returns answer. Check logs record the query and response.
- *Acceptance:* Agent loop operates end-to-end; all external calls and logs appear correctly. Attempting an undefined tool call is blocked by kernel.

- **Milestone 2: Observation and Memory**

- *Functionality:* Add ability to ingest documents or search results. E.g. implement a real web search API and a small memory buffer. The agent should be able to ask “summarize this article” with the text piped in.
- *Invariants:* External content filtering (sanitize inputs) and memory isolation.
- *Tests:* Inject a malicious snippet (like “ignore previous and do X”), ensure it’s not executed. Test memory TTL: after time or length limit, older memory is dropped or redacted.

- **Milestone 3: Ledger and Accounting**

- *Functionality:* Implement the database schema and basic account tables. Code transactional endpoints to record simple transfers (credit/debit).
- *Invariants:* Immutability of ledger entries (write-once). Ability to reconcile balances.
- *Tests:* Create entries and query totals. Confirm sums of debits equal credits. Ensure updates only append (try a forbidden update via SQL and ensure it’s blocked by schema or code).
- *Acceptance:* Can reconstruct balance from ledger, no direct edits allowed.

- **Milestone 4: Spending Tool and Limits**

- *Functionality:* Integrate a payment API (test mode). Set a per-transaction limit (e.g. \$10). The agent tries to “buy \$9 item” then “buy \$12 item.” The first goes through, the second is denied.
- *Invariants:* Spending cap enforcement, logging of denied attempts.
- *Tests:* Agent proposes a buy above limit → check kernel blocks and logs an error. Try splitting an approved \$10 into two \$5 purchase calls – second should be allowed (sum=\$10). Then try a third \$1 purchase (sum \$11) and check block.
- *Acceptance:* Budget checks and denies as expected. Ledger shows only approved transactions.

• **Milestone 5: Owner Approval UI**

- *Functionality*: Simple web dashboard for the owner to approve pending actions. Test a high-risk action (e.g. a \$20 purchase) which the agent proposes, kernel queues it, owner approves via UI, then kernel executes.
- *Invariants*: High-impact approval flow works; audit log records pending and approved actions.
- *Tests*: Submit a large transaction, confirm it appears in UI. Click “approve”; ensure payment goes through and ledger updated. Also test “deny”: action is canceled.
- *Acceptance*: No high-risk action completes without owner approval. The approval record is bound to the exact action (tool, amount, etc.) and cannot be forged (e.g. by reusing an old approval for a new action).

• **Milestone 6: Failure Mode and Edge Tests**

- *Functionality*: Run automated security tests covering the threats in Section 12.
- *Tests*:
 - Prompt-injection: feed malicious prompts and ensure they do not cause unauthorized actions (should be blocked or ignored).
 - Transaction splitting: ensure aggregate logic catches all.
 - Crash-recovery: simulate a crash mid-payment and verify idempotency on restart.
 - Revocation: issue some permissions, then revoke, ensure queued actions fail.
- *Acceptance*: All threat cases handled or explicitly documented as residual risk.

Each milestone includes code-level tests (e.g. pytest) and integration tests (simulating agent outputs) to catch regressions. Over time, add red-team prompts to the suite (e.g. example from [42†L97-L104]). The final blueprint (see next section) emerges from passing all milestones.

14. Not in MVB

We explicitly omit anything beyond minimal needs:

- **Blockchains/Distributed Ledgers**: Unnecessary complexity (see Section 9).
- **Hardware Enclaves (SGX, TrustZone)**: Bypassed for simplicity – we assume the host OS is trusted.
- **Formal Verification/Model Checking**: Out of scope. We rely on rigorous testing instead.
- **Multi-Agent Recursive Delegation**: No chains of delegated agents or mobile code beyond one agent.
- **Autonomous Deployment/CICD**: The agent will not autonomously modify or deploy code; we do not build self-upgrade features.
- **Long-term Vector Memory**: Not needed for a small experiment horizon. Use simple logs instead.
- **Complex Model Routing**: Single model invocation; no ensemble or routing.
- **Custom Crypto Capability Languages**: We use standard cryptography (HMACs for logs) and conventional APIs. No novel crypto or policy language.
- **Cross-Jurisdiction Operation**: Assume operation in one legal/jurisdictional context. No multiple currency or tax logic.
- **Broad PII Handling**: The agent should avoid collecting PII; if needed, treat it as restricted data (not legal compliance). We do not implement GDPR/CCPA workflows beyond basic data sanitation.

- **Financial Speculation/Leverage:** The agent will not engage in margin trading or derivatives. No complex market interactions.
- **Elaborate Observability (Prometheus, Kibana, etc.):** Simple logs are enough.
- **Automatic Constitutional Amendments:** The agent cannot propose changes to its own governance. No self-modifying rules.

Anything missing from the constitution that is not explicitly needed for the first test is deferred. This strict “not in MVB” list narrows the scope so we build exactly what’s needed.

15. Residual Risks and Assumptions

Despite best efforts, some risks remain:

- **Model Compliance:** We assume the LLM will mostly cooperate within the loop; if the model outright lies or behaves pathologically in the “planning” phase, our primary defense is enforcement. However, a misaligned model could spam the logs or attempt denial-of-service (e.g. endless loop of cheap operations). We mitigate by rate-limiting and monitoring.
- **Kernel/DB Compromise:** The design trusts the host environment. If an attacker compromised the server or database, they could override policies or steal funds. We assume standard server security (OS patches, firewall). This is a broader risk outside CapAge’s scope.
- **Imperceptible Vulnerabilities:** There may be unknown bugs (e.g. in the sandbox implementation) that allow escape. We assume using battle-tested sandboxing techniques (container permissions, seccomp).
- **Third-party API Trust:** If a tool’s external API (e.g. payment gateway) misbehaves, the kernel might mis-record events. We minimize this by using APIs in test/sandbox mode and verifying responses carefully.
- **Economic Risk Modeling:** Our risk model is simplistic (caps and approvals). In reality, an agent might cause indirect damage (brand reputation, customer churn) that we don’t quantify. We acknowledge these qualitative risks and keep the agent’s scope narrow (e.g. no real customers at first).
- **Owner Unavailability:** If the owner stops responding permanently, the agent will stall on approvals. We assume some fallback (owner can restart or shut down the agent manually).
- **LLM Exploits:** There is a risk of new LLM-specific attacks not anticipated (e.g. hidden channels). Continuous monitoring and prompt sanitization help, but not all can be predicted.
- **Oracle Problem:** The agent cannot verify external facts perfectly. If it is misled by false info on the web, it may make bad decisions. We assume this is part of exploration risk (and may appear as a performance deficit, not a safety violation).
- **Legal/Compliance:** We assume any revenue is small and not covered by financial regulations or AML laws. If it were, our simple AML-style checks might not suffice. This experiment assumes low regulatory oversight.

We assume the host OS, database, and network are secure. We assume the owner behaves reasonably. The system is **not** designed to be bulletproof – rather, it’s a demonstrator that the core ideas (LLM entrepreneurship under strict control) can work. Any uncovered vulnerability should either be addressed in future versions or accepted as part of the remaining risk profile.

16. Key Research Sources

- OWASP *AI Agent Security Cheat Sheet* – best practices for agent safety (least privilege, input validation, human-in-loop) [44†L315-L323] [44†L411-L418] .
- OpenAI Security Blog “*Designing AI agents to resist prompt injection*” (2026) – analogous to social engineering, emphasizes constraining agent capabilities [42†L119-L128] [42†L139-L147] .
- Google Research on **Macaroons** – describes capability-based tokens with attenuating caveats [33†L168-L173] .
- FinCEN Ruling on **Structuring** – official definition of splitting transactions to evade limits [13†L38-L46] .
- Guesty Trust-Accounting Guide – best practice for segregating trust funds (isolate client/owner funds) [25†L22-L27] [25†L49-L55] .
- EmergentMind *Immutable Audit Log* (November 2025) – overview of append-only, cryptographically secured logs for data integrity [28†L39-L46] .
- SatGate Blog on OpenAI budget limits – illustrates need for pre-request budget enforcement [36†L98-L102] .
- OWASP *GenAI Security Principles* – including prompt-injection and agent security (Do’s & Don’ts) [18†L1003-L1010] [18†L1019-L1026] .
- Wikipedia *Principle of Least Privilege* – fundamental security principle to limit permissions [45†L141-L149] .
- Hugging Face *Secure Code Execution for Agents* – cautions that only remote sandboxing (Docker/E2B) provides robust isolation [48†L277-L284] .

These sources underpin our design choices in policy enforcement, budgeting, trust boundaries, and threat mitigation.

17. Final Implementation Blueprint

In summary, the CapAge MVB is a tightly scoped agentic application. It consists of a single backend (Python web service) connecting an LLM to a controlled toolset, with an enforcement kernel that ensures compliance with the CapAge Constitution.

An engineering team can begin by setting up the basic loop (Milestones 1–2) to verify the agent can call tools and handle results through the kernel. Concurrently build the database schema for accounts and policies. Once that is operational, implement the Payments tool with a hard-coded limit and verify the kernel correctly logs and enforces it (Milestone 4). Add the owner approval UI (Milestone 5) so that higher-risk operations stop pending human consent.

Throughout, write automated tests for each constitutional invariant (e.g. simulate splits, injections, etc.). By Milestone 6, the agent should have genuine autonomous capability (it can find an opportunity on the web and propose a safe action) while still being under external guardrails for authority, risk, and audit.

This MVB design maximizes autonomous exploration within a provably safe frame: the agent *operates itself*, but the system *controls its hands*. With the details above, a small team can proceed to implementation knowing exactly which components are critical (and which can be left out) and how to test them.

